

TECH CUP - Engineering Assignment

¡Bienvenidos al Sprint #3 del desafío técnico de TechCup! Esta semana continuamos con el desarrollo progresivo del proyecto. Durante este sprint se introducirán nuevas implicaciones que deben ser analizadas e integradas en la solución actual. El objetivo de este ejercicio es simular un entorno real de ingeniería, donde los sistemas evolucionan sprint tras sprint y el equipo debe adaptar su solución de acuerdo al feedback obtenido.

CONSIDERACIONES DEL TRABAJO EN EQUIPO

Para el desarrollo del proyecto deberán cumplir con las siguientes dinámicas de trabajo:

1. Planning:

Cada equipo deberá realizar una reunión inicial de planificación (**Planning**) al inicio de cada sprint, con una duración definida por el equipo según su necesidad.

En esta reunión se debe:

- Definir el **alcance del sprint** (qué se va a desarrollar).
- Priorizar y distribuir las **tareas entre los integrantes**.
- Estimar el esfuerzo de las actividades.
- Identificar posibles **riesgos o bloqueos iniciales**.
- Alinear expectativas sobre entregables y tiempos.

NOTA: Deben dejar evidencia de la reunión que incluya:

- a. Resumen de lo discutido.
- b. Integrantes que participaron.
- c. Tareas asignadas y responsables.
- d. Decisiones tomadas.

2. Dailies:

Cada equipo deberá realizar **reuniones diarias (Daily)** de aproximadamente **15 minutos**. En estas reuniones cada integrante debe responder:

- Qué hice ayer con respecto a mis asignaciones
- Qué haré hoy
- Qué bloqueos tengo

NOTA: Si no pueden realizar la reunión virtual, pueden hacerla presencial, pero deben dejar evidencia escrita que incluya:

- resumen de lo discutido por todos los integrantes

- integrantes que asistieron
- decisiones tomadas

3. Comunicación del Equipo:

El equipo debe tener un canal de comunicación activo para coordinar el proyecto, puede ser mediante:

- **Microsoft Teams**
- **WhatsApp**

El docente debe estar **incluido en este canal** para poder observar el proceso de colaboración del equipo.

4. Organización del Equipo:

Cada sprint deberán compartir:

- **Organigrama del equipo:** Debe incluir sus roles asignados, este organigrama puede cambiar de sprint a sprint.
- **Disponibilidad del equipo:** Cada integrante debe indicar: horarios disponibles y posibles restricciones de tiempo según sus actividades académicas o laborales

Esto permitirá planear correctamente el trabajo del sprint.

ACTIVIDADES DEL SPRINT #3

Durante este sprint deberán realizar las siguientes actividades

1. IDENTIFICACIÓN Y SOLUCIÓN DE LA DEUDA TÉCNICA SPRINT #2:

Antes de continuar con el desarrollo del sistema, el equipo debe revisar el estado actual del proyecto junto con los comentarios de feedback que se presentaron en la sesión de revisión por parte del profesor.

Entre las actividades se incluyen:

- Revisar los requerimientos establecidos y actualizarlos con el feedback recomendado.
- Revisar el análisis de requerimientos y actualizarlo con los comentarios realizados.
- Identificación de inconsistencias en las definiciones de los artefactos entregados:
 - Mockups del sistema - Correcciones de forma y de estructura, navegación, y flujos por roles.
 - Manual de identidad visual - Correcciones de logotipos, colores utilizados y explicación técnica como soporte.
 - Repositorio Back End - Readme técnico

- Repositorio Front End - Generación y correcciones en el Readme tecnico
- Diagramas del sistema - Correcciones:
 - Diagrama de Contexto
 - Diagrama de Clases
 - Diagrama de componentes General
 - Diagrama de componentes Específico
 - Diagramas de Secuencia por operaciones (Implementadas y por Implementar)
- Detección de deuda técnica - Puntos faltantes si no fueron desarrollados
 - Funcionalidades implementadas en BackEnd
 - Set de pruebas y su estructura
 - Implementación de Análisis estático de pruebas y Cobertura de código
 - Organización de la paquetería de manera adecuada y su documentación con Swagger

2. REFINAMIENTO DEL BACKLOG:

Una vez identificada la deuda técnica, el equipo deberá organizar el trabajo del proyecto dentro de JIRA:

A. Product Backlog:

- a. Dentro de cada una de las ÉPICAS establecidas, genere las tareas necesarias asociadas a la deuda técnica presentada en el Sprint #2

NOTA: Cada deuda técnica identificada debe registrarse como una feature o tarea independiente dentro del backlog atada a la épica correspondiente.

- b. Genere un feature asociado a la Implementación de la persistencia relacional dentro del sistema, recuerde que tiene que fragmentarlo en las tareas necesarias:
 - i. Implementación de la dependencia dentro del pom.xml
 - ii. Creación de las configuraciones de conexión con la base de datos dentro del application.yaml
 - iii. Implementación de la persistencia dentro de las funcionalidades que ya fueron creadas e implementadas en Sprint #1 y Sprint #2
 - iv. Implementación de la persistencia dentro de las funcionalidades para el Sprint #3
- c. Para las funcionalidades necesarias para el sistema, tanto las que ya fueron implementadas en sprint #1 y sprint #2 como las que están en el backlog, se le sugiere que para las tareas de backend agregue las siguientes tareas de ejemplo:
 - i. Creación de las entidades
 - ii. Creación de los repositorios
 - iii. Creación de Mappers - Modelo a Entidad/DAO
 - iv. Creación de Mappers - Entidad/DAO a Modelo

- v. Pruebas de los Mappers de la capa de persistencia
- vi. Manejo de las Excepciones desde Base de datos hacia los servicios:
 - 1. Personalizadas
 - 2. De negocio
 - 3. Genéricas
- vii. Inyección de los repositorios dentro de los servicios
- viii. Evidencia de pruebas funcionales de conectividad e implementación de la persistencia

NOTA: Son sugerencias para que el equipo pueda iniciar a trabajar correctamente, si necesita más tareas que cumplan con la creación y ejecución correcta de dicha funcionalidad se pueden agregar.

- d. Para las feature de arquitectura incluya:
 - i. Diagrama de Entidad Relación - Donde establezcan el modelo de cómo van a manejar el modelo relacional de Base de datos y la normalización de sus tablas utilizadas en el proyecto.
 - ii. Para el Front End en la épica establecida deben ir las funcionalidades visuales del sistema, para este sprint **ÚNICAMENTE** deben continuar con las sub tareas relacionadas con los MockUps.

3. REFINAMIENTO DEL SET DE PRUEBAS:

A partir de lo visto en clase con respecto a la cobertura y análisis de código deberán agregar a la definición de los escenarios de prueba para cada funcionalidad **TANTO LAS YA IMPLEMENTADAS como LAS QUE VAN A IMPLEMENTAR:**

- Evidencias de los registros de cobertura a nivel de Líneas, Ramas, Métodos y Clases con Jacoco
- Evidencia de la ejecución con SonarQube del análisis estático de código
 - Si los calificativos son menores a B tiene que realizar las tareas correspondientes para mejorar y subir sus puntuaciones.

Estos deben agregarse a JIRA como evidencia dentro de cada feature.

4. IMPLEMENTACIÓN DE SEGURIDAD API:

A partir de lo visto en clase con respecto a temas de seguridad: Autorización, Autenticación e Integridad implemente dentro de sus funcionalidades lo siguiente:

- a. Cree dos features especializados en el manejo de la Autenticación implementando JWT (usuarios internos) y Oauth2 con google (usuarios externos) y por cada una:
 - i. Implemente las dependencias necesarias en el pom.xml
 - ii. Implemente las configuraciones para que su aplicación pueda implementarlo
 - iii. Cree los controladores especializados para el tema de Autenticación

- iv. Genere los servicios necesarios para manejar la generación de los tokens y el uso de los mismos en las peticiones por operación
- v. Establezca el TTL de cada token (Recomendación - 1 hora)
- vi. Realice las pruebas funcionales sobre la implementación de la Autenticación
- vii. Valide que con las funcionalidades implementadas le permita utilizar correctamente el token generado:
 - 1. Pruebas de Flujo correcto
 - 2. Pruebas de Flujo de token expirado
 - 3. Pruebas de Flujo de token invalido
 - 4. Pruebas de Flujo sin enviar token
- b. Cree un feature especializado en el manejo de la Autorización, donde establezca:
 - i. La implementación de los Roles dentro del sistema
 - ii. Establezca qué operaciones puede realizar cada rol
 - iii. Implemente la autorización dentro de cada uno de los controladores creados
 - iv. Verifique y valide con pruebas funcionales:
 - 1. Pruebas por tipo de rol utilizando las operaciones permitidas
 - 2. Pruebas en funcionalidades con roles que no tienen permisos sobre las mismas.
- c. Cree un feature especializado en el manejo de la Integridad, donde establezca:
 - i. El manejo de SSL/TLS para volver su aplicación http una aplicación https
 - ii. Habilite el uso de CORS dentro de su aplicación

5. CONSTRUCCIÓN DEL SPRINT BACKLOG SPRINT #3:

- A. Después de organizar correctamente la planeación del trabajo, Identifique cuáles tareas son más prioritarias, cuáles tareas tienen correlación directa, que tarea puede bloquear a otra o es derivada de otra.
- B. El líder de la semana anterior tiene que cerrar el Sprint #2, resumiendo en el cierre que se logró y que quedó como deuda técnica para el Sprint #3.
- C. Del Backlog Seleccione las actividades que planean realizar y desarrollar en este sprint, recuerde que tiene que ser concordante con la matriz de trazabilidad que realizaron en Sprint #1 - Adicional recuerde la cantidad de semanas de trabajo, distribuyan bien la carga laboral.
- D. Arrastre cada una de esas tareas al Sprint Backlog del Sprint #3.
- E. Una vez seleccionadas, realicen la correcta estimación de las tareas, recordando usar Planning Poker para la definición, Fibonacci para los puntajes y recordando que la carga máxima por persona. - Recuerde el balance entre todos los integrantes del equipo.
- F. Finalmente deberán Iniciar el Sprint en JIRA y definir el objetivo para este SPRINT #3.

6. CREACIÓN DE DIAGRAMAS DE ARQUITECTURA:

El equipo deberá continuar con el diseño de la arquitectura del sistema. Deben construir los siguientes diagramas y agregarlos como índices dentro del README de su backend:

- **Diagramas de Entidad Relación:** Enfocado en el detalle de cómo interactúan los componentes de base de datos y sus relaciones.

Recuerde que todos los diagramas deben tener una descripción y explicación clara.

7. IMPLEMENTACIÓN DE CÓDIGO:

Durante este sprint deberán continuar con el desarrollo a nivel de código de sus funcionalidades, utilizando una arquitectura MVC.

Deben implementar:

- Por funcionalidad escogida y tareas a realizar este sprint, deberán crear todo el flujo desde el controlador hasta la creación de los servicios, utilidades y validadores.
- Pruebas de acuerdo al set de pruebas que surge para esa funcionalidad que se va a trabajar.
- Manejo de errores.
- Implementación de la persistencia.

8. SPRINT REVIEW:

Al finalizar el sprint deberán realizar una presentación del trabajo realizado, la cual será expuesta en la sesión de laboratorio por cada líder técnico. La presentación debe incluir:

- **Miembros del equipo y que rol tuvo cada uno esta semana**
- **Objetivo del sprint:** Qué buscaba lograr el equipo durante esta semana.
- **Distribución de tareas:** Cómo se repartió el trabajo dentro del equipo, Como quedo la matriz de trazabilidad y el sprint backlog para el SPRINT #2.
- **Métricas del Sprint:** Puntos asignados a cada integrante, Puntos logrados por cada integrante y puntos totales del sprint
- Demo funcional: Demostración del sistema funcionando y ejecutado con spring boot - Correcciones realizadas y Que lograron avanzar este sprint

9. SPRINT RETROSPECTIVE: Retroalimentación del Equipo

Finalmente el equipo debe reflexionar sobre el proceso respondiendo:

- ¿Qué hicieron bien?
- ¿Qué hicieron mal?
- ¿Qué mejorarían en el siguiente sprint?
- ¿A qué se comprometen para el próximo sprint?



ENTREGABLES:

Dentro del espacio de moodle deben enviar:

- **Link de su repositorio BackEnd**
- **Link de su repositorio FrontEnd**
- **Link de sus mockups con Figma**
- **Documento de Análisis de Requerimientos**
- **Documento de Manual de Identidad**
- **Presentación de Canva o Powerpoint de la retrospectiva y review**